

Material Complementar — Métodos Locais e Kernels

EST335 — Métodos Estatísticos para Machine Learning

Prof. Tiago Martins Pereira

Índice

Objetivo	1
1. Dados para os exemplos	2
2. KNN como regressão local constante	2
Implementação simples	3
3. Escolha de hiperparâmetros por validação cruzada	4
4. Nadaraya–Watson	5
5. KNN versus kernel	6
6. Regressão polinomial local	7
7. Variáveis irrelevantes e distância	8
8. Kernel Ridge	9
9. Kernel Gaussiano	9
10. O que observar na prática	10

Objetivo

Este material apresenta um roteiro prático para estudar **métodos locais, suavização e kernels** em R. Os exemplos usam dados simulados diferentes dos dados da Atividade 2 e não fornecem sua solução.

A ideia central é perceber que métodos não paramétricos substituem uma forma funcional global rígida por mecanismos que controlam a complexidade localmente.

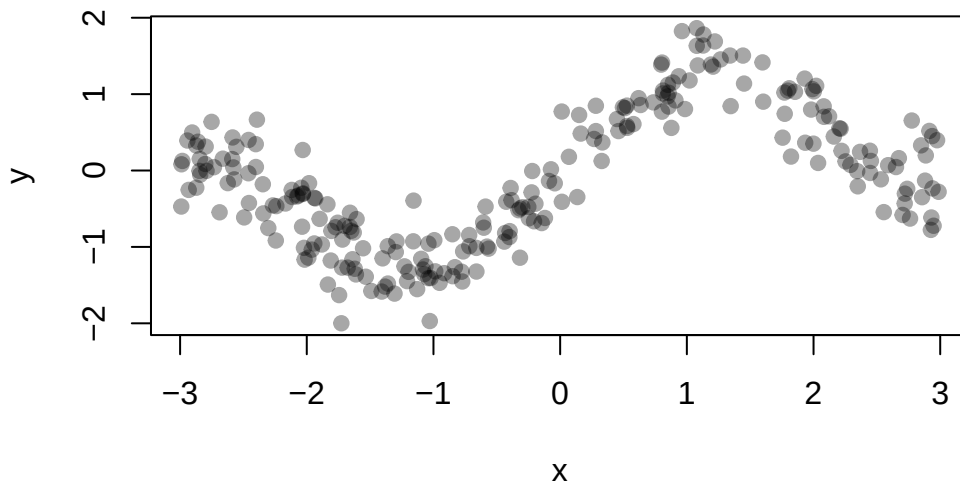
1. Dados para os exemplos

```
set.seed(335)

n <- 250
x <- runif(n, -3, 3)
y <- sin(1.5*x) + 0.25*x + rnorm(n, sd = 0.35)

dados <- data.frame(x, y)

plot(
  dados$x, dados$y,
  pch = 19, col = rgb(0,0,0,0.35),
  xlab = "x", ylab = "y"
)
```



2. KNN como regressão local constante

Para um ponto x_0 , o KNN usa os k vizinhos mais próximos:

$$\hat{g}(x_0) = \frac{1}{k} \sum_{i \in N_k(x_0)} y_i.$$

Essa média também resolve

$$\min_{\beta_0} \sum_{i \in N_k(x_0)} (y_i - \beta_0)^2.$$

Assim, KNN pode ser interpretado como um **ajuste local de grau zero**.

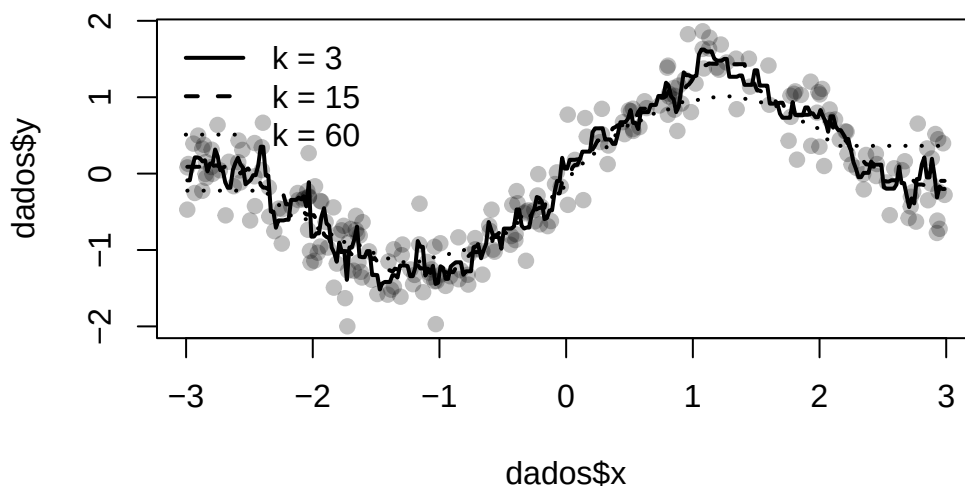
Implementação simples

```
pred_knn <- function(x_train, y_train, x_new, k) {
  sapply(x_new, function(x0) {
    ind <- order(abs(x_train - x0))[1:k]
    mean(y_train[ind])
  })
}

grid <- seq(min(x), max(x), length.out = 300)

p3 <- pred_knn(x, y, grid, k = 3)
p15 <- pred_knn(x, y, grid, k = 15)
p60 <- pred_knn(x, y, grid, k = 60)

plot(dados$x, dados$y, pch = 19, col = rgb(0,0,0,0.25))
lines(grid, p3, lwd = 2)
lines(grid, p15, lwd = 2, lty = 2)
lines(grid, p60, lwd = 2, lty = 3)
legend(
  "topleft",
  legend = c("k = 3", "k = 15", "k = 60"),
  lty = 1:3, lwd = 2, bty = "n"
)
```



Observe que k funciona como parâmetro de suavização:

$k \downarrow \Rightarrow$ modelo mais local e flexível,

$k \uparrow \Rightarrow$ modelo mais suave.

3. Escolha de hiperparâmetros por validação cruzada

O conjunto de teste deve representar dados ainda não utilizados. Portanto, k não deve ser escolhido minimizando o erro de teste.

Um procedimento adequado é:

1. separar treino e teste;
2. executar validação cruzada apenas no treino;
3. selecionar k ;
4. ajustar o procedimento escolhido;
5. avaliar uma única vez no teste.

O mesmo princípio será usado para h , λ e outros hiperparâmetros.

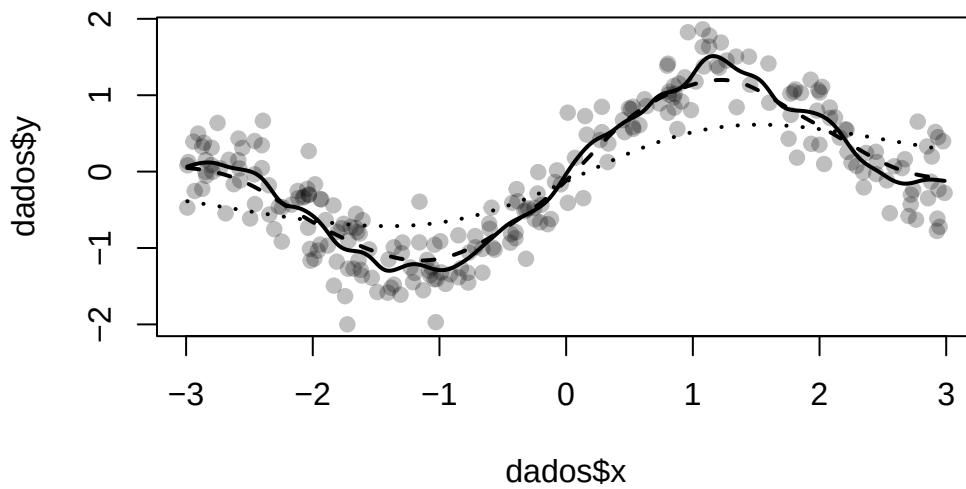
4. Nadaraya–Watson

O estimador é

$$\hat{g}_h(x_0) = \frac{\sum_i K\left(\frac{x_i - x_0}{h}\right) y_i}{\sum_i K\left(\frac{x_i - x_0}{h}\right)}.$$

Com kernel Gaussiano:

```
K_gauss <- function(u) {  
  dnorm(u)  
}  
  
nw <- function(x_train, y_train, x_new, h) {  
  sapply(x_new, function(x0) {  
    w <- K_gauss((x_train - x0)/h)  
    sum(w*y_train) / sum(w)  
  })  
}  
  
h1 <- nw(x, y, grid, h = 0.10)  
h2 <- nw(x, y, grid, h = 0.35)  
h3 <- nw(x, y, grid, h = 1.00)  
  
plot(dados$x, dados$y, pch = 19, col = rgb(0,0,0,0.25))  
lines(grid, h1, lwd = 2)  
lines(grid, h2, lwd = 2, lty = 2)  
lines(grid, h3, lwd = 2, lty = 3)
```



Aqui, ao contrário do KNN,

$h \downarrow \Rightarrow$ maior localidade,

enquanto

$h \uparrow \Rightarrow$ maior suavização.

5. KNN versus kernel

No KNN, uma representação simplificada dos pesos é

$$w_i(x_0) = \begin{cases} 1/k, & x_i \in N_k(x_0), \\ 0, & \text{caso contrário.} \end{cases}$$

No Nadaraya–Watson,

$$w_i(x_0) = \frac{K((x_i - x_0)/h)}{\sum_j K((x_j - x_0)/h)}.$$

Assim, o kernel permite que a importância das observações varie continuamente com a distância.

6. Regressão polinomial local

Em vez de assumir que a função é constante perto de x_0 , podemos aproximá-la localmente por

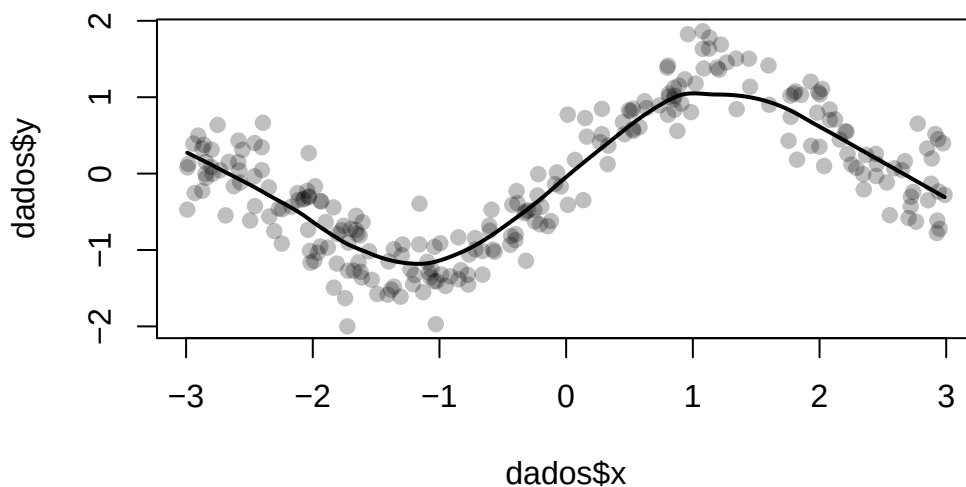
$$y_i \approx \beta_0(x_0) + \beta_1(x_0)(x_i - x_0).$$

O ajuste é obtido por mínimos quadrados ponderados:

$$\min_{\beta_0, \beta_1} \sum_i K\left(\frac{x_i - x_0}{h}\right) [y_i - \beta_0 - \beta_1(x_i - x_0)]^2.$$

Em R, `loess()` permite explorar essa ideia:

```
ajuste <- loess(  
  y ~ x,  
  data = dados,  
  degree = 1,  
  span = 0.35  
)  
  
pred <- predict(  
  ajuste,  
  newdata = data.frame(x = grid)  
)  
  
plot(dados$x, dados$y, pch = 19, col = rgb(0,0,0,0.25))  
lines(grid, pred, lwd = 2)
```



Uma vantagem importante do ajuste linear local é seu comportamento próximo às fronteiras.

7. Variáveis irrelevantes e distância

Considere que a resposta dependa principalmente de x_1 , mas adicionemos variáveis independentes $z_1, z_2, \dots$

Em métodos baseados em distância, a noção de “vizinho” passa a depender de todas as coordenadas utilizadas.

Com o aumento da dimensão, os pontos tendem a ficar mais esparsos no espaço:

$$d \uparrow \Rightarrow \text{vizinhanças locais tornam-se menos informativas.}$$

Esse fenômeno está relacionado à **maldição da dimensionalidade**.

Por isso, padronização e seleção de covariáveis são especialmente relevantes para KNN.

8. Kernel Ridge

Na regressão Ridge tradicional:

$$\hat{\beta} = \arg \min_{\beta} \left\{ \sum_i (y_i - x_i^\top \beta)^2 + \lambda \|\beta\|_2^2 \right\}.$$

Em um espaço de Hilbert com kernel, podemos considerar

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \left\{ \frac{1}{n} \sum_i (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2 \right\}.$$

O primeiro termo controla o ajuste aos dados. O segundo controla a complexidade da função.

Pelo Teorema do Representante, a solução pode ser escrita como

$$\hat{f}(x) = \sum_{i=1}^n \alpha_i K(x_i, x).$$

Assim, não é necessário trabalhar explicitamente com todas as coordenadas do espaço de características induzido pelo kernel.

9. Kernel Gaussiano

Um kernel muito utilizado é

$$K(x, x') = \exp(-\gamma \|x - x'\|^2).$$

O parâmetro γ controla a escala de similaridade, enquanto λ controla a regularização.

Portanto, existem dois mecanismos distintos:

$\gamma \rightarrow$ geometria/localidade do kernel,

$\lambda \rightarrow$ penalização da complexidade.

10. O que observar na prática

Ao comparar os métodos, não se limite a perguntar qual produz o menor RMSE.

Pergunte também:

- como a curva muda quando o parâmetro de suavização muda?
- o que acontece nas fronteiras?
- como o método reage a regiões com poucos dados?
- o que ocorre quando covariáveis irrelevantes entram na distância?
- como o desempenho de validação se relaciona ao desempenho de teste?
- quais parâmetros controlam a complexidade em cada método?

A comparação dessas respostas é mais importante para a compreensão estatística dos métodos do que a simples ordenação das métricas.